功能链路分析报告 V1.0

易工单系统

XiaoBai Easy WorkOrder System

50	3	140	10
已实现功能点	子系统	单元测试	数据库迁移版本

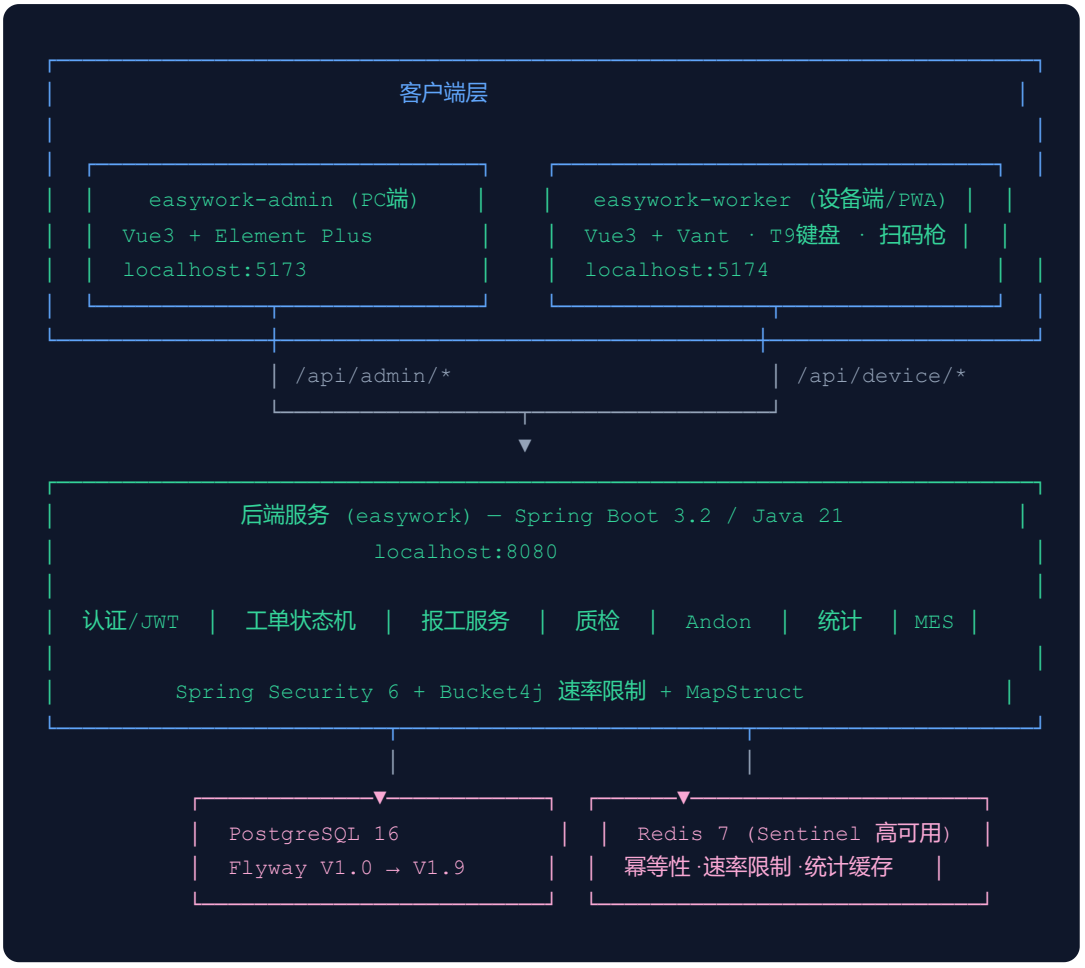
报告生成时间：2026-03-18 | 调研方式：静态代码分析 + 实际页面截图

报告目录

01	系统架构总览	三端分离架构、技术栈、部署方式
02	用户故事 A：工单生命周期	创建 → 派工 → 开工 → 报工 → 质检 → 完成
03	用户故事 B：工人操作链路	工人端扫码、开工、报工、撤销、批量操作
04	用户故事 C：质检与返工链路	质检录入、通过/失败/报废、返工循环
05	用户故事 D：Andon 呼叫链路	工人触发 → 管理员响应 → 处理完成
06	管理后台功能全览	用户、班组、统计、审计、MES集成
07	关键风险与缺口	架构风险、代码质量、功能缺口
08	架构改进建议	Agent/MCP/Skills 视角的落地建议

01 系统架构总览

三端分离架构 · Spring Boot 后端 · Vue3 双前端 · PostgreSQL + Redis 数据层



技术栈一览

分层	技术选型	版本	核心用途
后端框架	Spring Boot	3.2.3	Web + 安全 + 自动配置
数据访问	MyBatis-Plus	3.5.7	ORM + 乐观锁 + 逻辑删除
认证	Spring Security 6 + JJWT	0.12.5	JWT 无状态认证
数据库	PostgreSQL	16	主数据存储，Flyway 版本管理

分层	技术选型	版本	核心用途
缓存	Redis 7 + Sentinel	—	幂等性、速率限制、本地 Caffeine 缓存
管理端 UI	Vue 3 + Element Plus	3.5 / 2.13	PC 端管理后台
工人端 UI	Vue 3 + Vant	3.5 / 4.9	移动端/手持设备，支持 T9 键盘
图表	ECharts	6.0	统计看板可视化
工序图	Vue Flow	1.48	工序依赖有向图（DAG）
离线支持	Service Worker + IndexedDB	—	断网排队、联网重放

工单状态机核心规则

生产型工单 (PRODUCTION)

未开始 → 开工 → 已开始 → 报工 → 待质检 → 质检通过 → 质检通过 → 完成 → 已完成

待质检 → 质检失败 → 质检失败 → 返工重开 → 待质检 (循环直至通过)

待质检 → 报废 → 已报废 (终态)

简单型工单 (INSPECTION / TRANSPORT / ANDON)

未开始 → 开始 → 已开始 → 完成 → 已完成

02 用户故事 A：工单生命周期

从管理员创建工单，到最终完成的完整链路 · 角色：管理员

A1. 管理员登录

1 访问系统

管理员打开浏览器访问管理端，系统检测未登录，自动跳转登录页

▼

2 输入员工号 + 密码

输入员工号（如 ADMIN001）和密码，点击登录

▼

3 获取 JWT Token

后端验证通过，返回 JWT Token，前端存入 localStorage，跳转看板

接口

`POST` /api/auth/login · 参数: {employeeNumber, password}

速率限制：10次/60秒（Bucket4j）· CORS 白名单配置

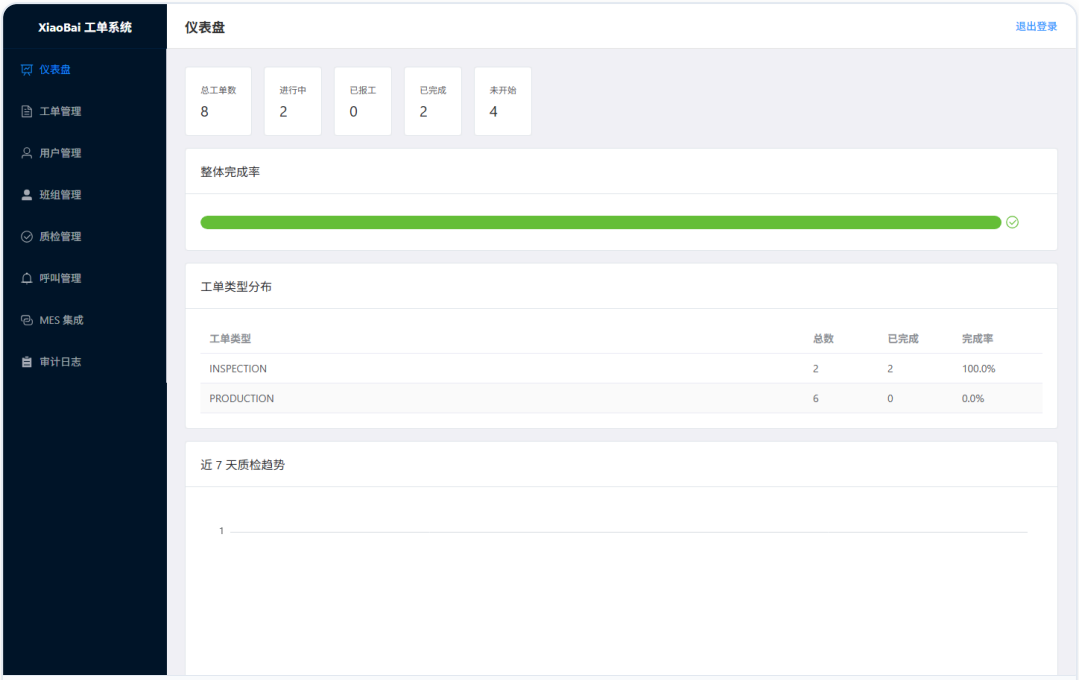
管理端登录页 — 输入员工号和密码

02 用户故事 A（续）：创建工单 & 统计看板

登录后首先看到统计看板，然后进入工单管理模块

A2. 统计看板（登录后默认页）

登录成功后自动跳转 /dashboard，展示：工单总量、完成率、质检通过率、质检趋势折线图（近7天/N天）



统计看板 — ECharts 可视化（工单数量、质检趋势）

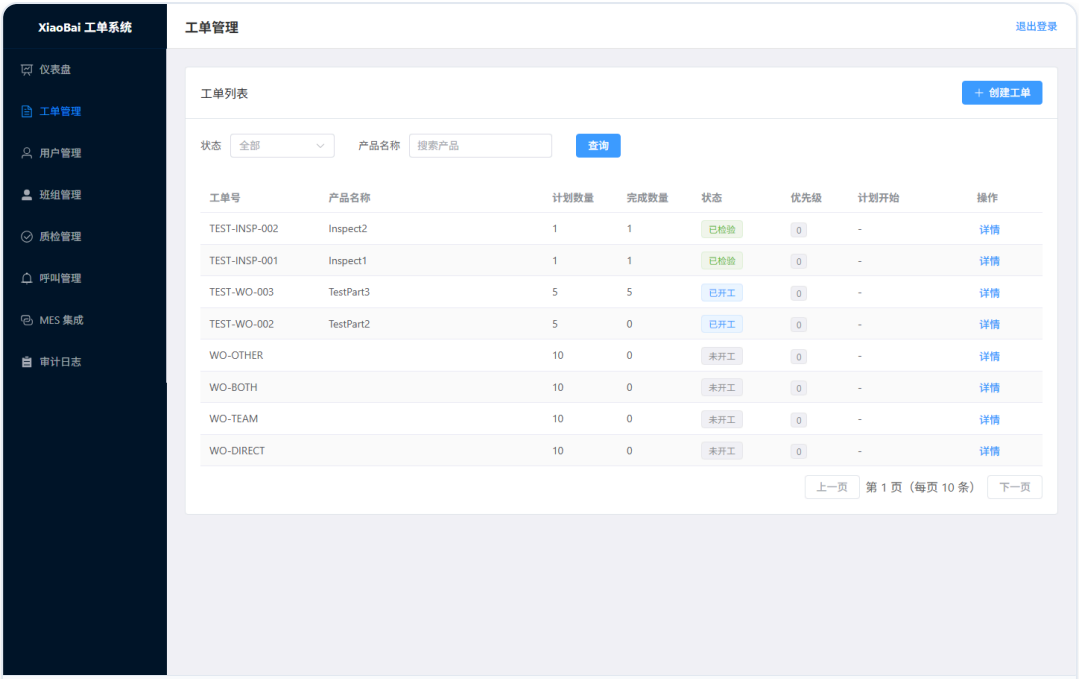
接口

- **GET** /api/admin/statistics/dashboard
- **GET** /api/admin/statistics/inspection-trend?days=7

后端 StatisticsService 使用 Caffeine 本地缓存，减少数据库聚合查询压力

A3. 工单列表

展示全部工单，支持按状态（未开始/已开始/待质检/已完成等）和产品名筛选，分页显示。


XiaoBai 工单系统
工单管理
退出登录
仪表盘
工单管理
用户管理
班组管理
质检管理
呼叫管理
MES 集成
审计日志
工单列表
+ 创建工单
状态
全部
产品名称
搜索产品
查询
工单号 产品名称 计划数量 完成数量 状态 优先级 计划开始 操作
TEST-INSP-002 Inspect2 1 1 已检验 0 - 详情
TEST-INSP-001 Inspect1 1 1 已检验 0 - 详情
TEST-WO-003 TestPart3 5 5 已开工 0 - 详情
TEST-WO-002 TestPart2 5 0 已开工 0 - 详情
WO-OTHER 10 0 未开工 0 - 详情
WO-BOTH 10 0 未开工 0 - 详情
WO-TEAM 10 0 未开工 0 - 详情
WO-DIRECT 10 0 未开工 0 - 详情
上一页
第 1 页（每页 10 条）
下一页

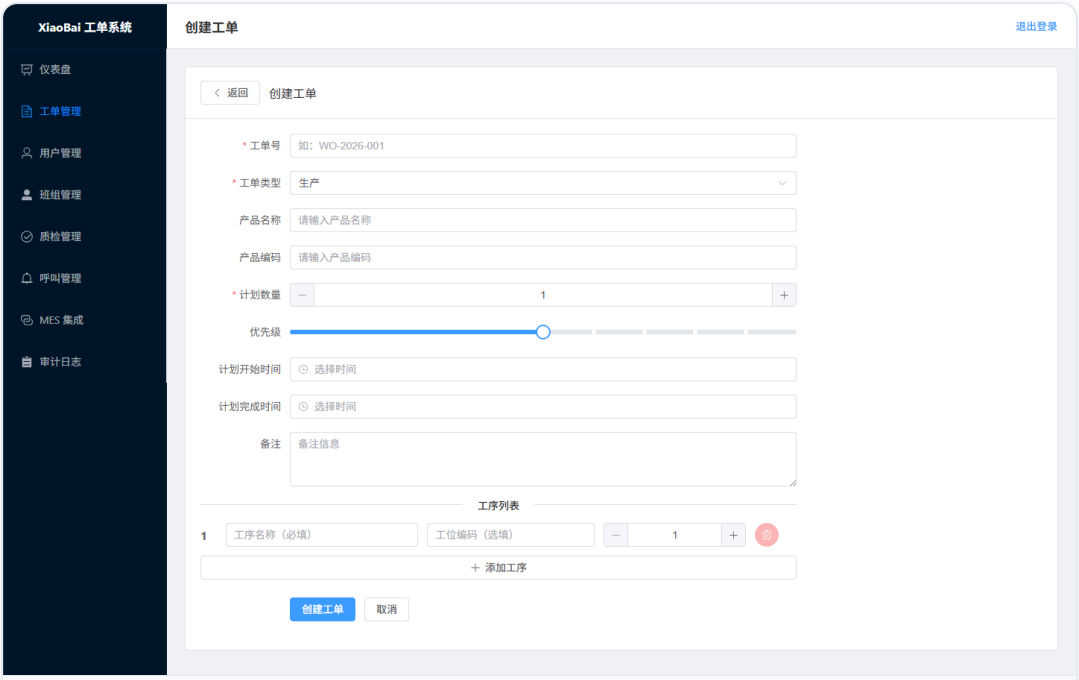
工单列表 — 状态筛选 + 分页

02 用户故事 A（续）：创建工单 & 工单详情

新建工单并查看工序依赖关系图

A4. 创建工单

支持4种工单类型：生产(PRODUCTION)、质检(INSPECTION)、搬运(TRANSPORT)、Andon。每种类型对应不同状态流转规则。



创建工单页 — 选择类型、填写产品信息、添加工序

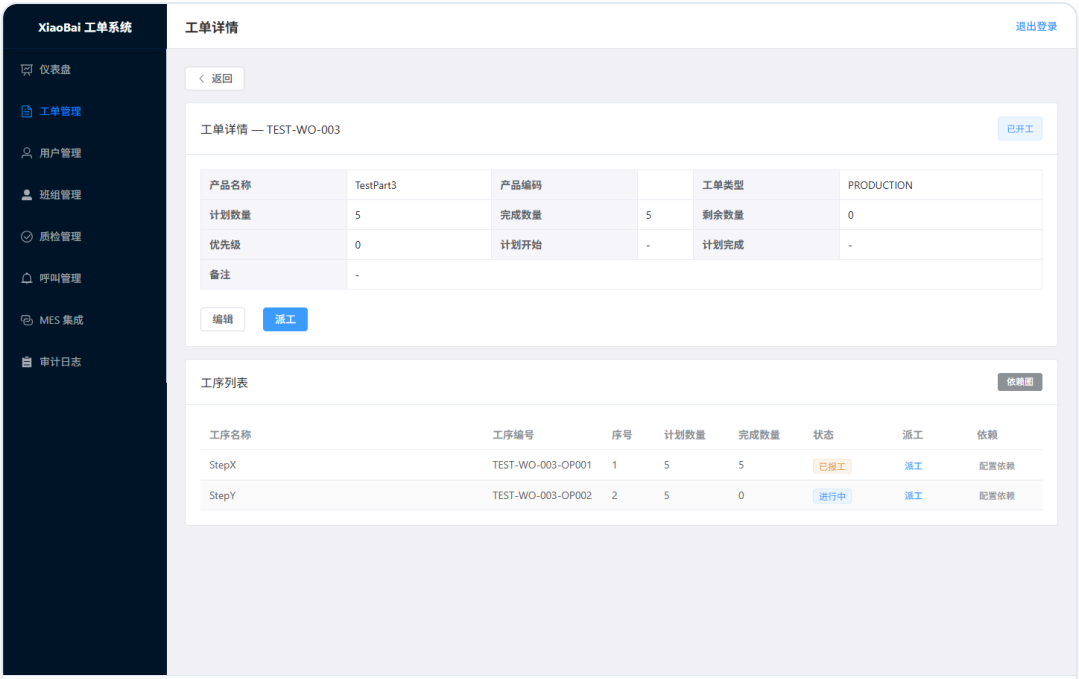
接口 + 后端文件

`POST` /api/admin/work-orders · {productName, quantity, type, operations[]}

AdminWorkOrderController → WorkOrderService → WorkOrderMapper → work_orders + operations 表
创建时工单状态自动设为 NOT_STARTED，operations 各自也初始化为 NOT_STARTED

A5. 工单详情 + 工序有向图

工单详情页展示：工序列表、每道工序当前状态、已完成/计划数量。若配置了工序依赖关系，还会渲染 DAG 有向图（Vue Flow + Kahn 拓扑排序）。



工单详情 — 工序状态 + 操作按钮（派工/质检/完成）

03 用户故事 B：工人操作链路

工人在车间手持设备上的完整操作流程 · 角色：工人

B1. 工人登录（T9 键盘）

工人端专为手持设备/PDA 设计，密码输入支持 T9 九键键盘，方向键可在工单列表中导航，数字快捷键执行操作。

工人端登录页 — T9 九键键盘输入密码

工人端工单列表 — 方向键导航 + 焦点高亮

接口 + 硬件适配

POST /api/device/login · {employeeNumber, password}

> useHardwareInput.js：50ms 阈值识别扫码枪输入 · 方向键 ↑↓ 导航工单/工序列表
> 数字快捷键：1=开工 / 2=报工 / 3=质检 / 4=撤销 / 5=返工（工单详情页）

B2. 工单详情（工人视角）

工人在工单详情页可看到本工单的所有工序及其状态，点击工序执行相应操作。键盘快捷键覆盖所有操作，无需触屏。

TEST-WO-003
100%
产品名称
工单类型
PRODUCTION
计划数量
5
完成数量
5
剩余数量
0
优先级
0
状态
已开工
工序列表
StepX
已报工
计划：5 完成：5
撤销报工
返工
StepY
进行中
计划：5 完成：0
报工
发起呼叫

工人端工单详情 — 工序操作（开工/报工/撤销/质检）

03 用户故事 B（续）：扫码操作 & 批量操作

扫码枪快速开工/报工 · 批量操作支持幂等性

B3. 扫码开工/报工

工人将扫码枪扫描工序/工单条码，系统自动识别条码类型并执行相应操作（开工或报工），无需手动查找工序。

1 扫描条码

扫码枪扫描贴在工序卡/工单上的条码，系统在 50ms 内完成输入收集

▼

2 条码解析

ScanService 自动判断：工单条码 or 工序条码，找到对应记录

▼

3 执行操作

根据当前操作模式（开工/报工），自动触发 startWork 或 reportWork

▼

4 返回结果

显示操作成功/失败提示，刷新工序状态

扫码操作
开工模式
报工模式
100%
输入工单条码
条码
请输入或扫描工单号
扫码开工
扫码报工

扫码页 — 扫码枪/手动输入条码，切换开工/报工模式

接口

POST /api/device/scan/start · {barcode}

POST /api/device/scan/report · {barcode}

B4. 批量开工/报工（幂等性保证）

工人可多选工序后批量执行开工或报工。每次批量操作生成 UUID 作为幂等 Key，网络重传不会导致重复操作。

幂等机制

请求头携带 `Idempotency-Key: {UUID}`
后端 IdempotencyService 将结果缓存到 Redis，TTL 30分钟
重复提交时直接返回缓存结果，不执行二次操作

离线支持

断网时操作自动进入 IndexedDB 离线队列（offlineQueue.js）
Service Worker 监听网络恢复，自动重放队列中的请求
重放结果返回 {processed, failed, skipped}

04 用户故事 C：质检与返工链路

工序报工后进入质检环节，质检结果触发通过/失败/报废三种分支

C1. 质检管理（管理端）

管理员或质检员在管理端查看"待质检"状态的工单，录入质检结果。

1 工单进入待质检

工人完成所有工序报工后，工单状态自动变为 REPORTED（待质检）

▼

2 质检录入

质检员在管理端 /inspection 页面，选择工单，填写检查数量、合格数量、缺陷数量、缺陷原因

▼

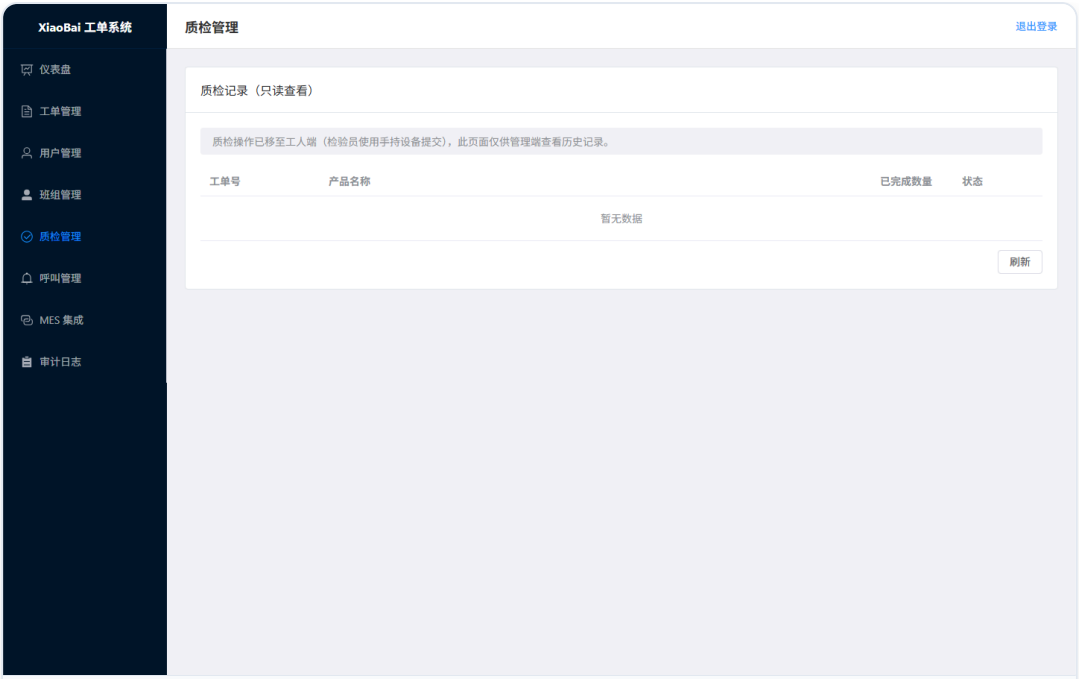
3 质检结果判定

选择 PASSED / FAILED / SCRAPPED 三种结果之一

▼

4 状态流转

PASSED → INSPECT_PASSED（可完成）/ FAILED → INSPECT_FAILED（需返工）/ SCRAPPED → 终态



质检管理页 — 待质检工单列表 + 质检结果录入表单

接口 + 后端文件

`POST` /api/admin/inspections · {workOrderId, inspectionResult, inspectedQuantity, ...}

InspectionController → InspectionService → inspection_records 表
质检结果同步触发 WorkOrderStateMachine 状态转换

C2. 返工流程

质检失败后，管理员可通过"重新打开"操作将工单状态从 INSPECT_FAILED 切回 REPORTED，工人重新执行工序并报工，直至质检通过。

待质检 → 质检失败 → **质检失败** → 重新打开 → **待质检**
→（重新报工循环）

接口

`PUT` /api/admin/work-orders/{id}/reopen

`PUT` /api/admin/work-orders/{id}/complete

05

用户故事 D：Andon 呼叫链路

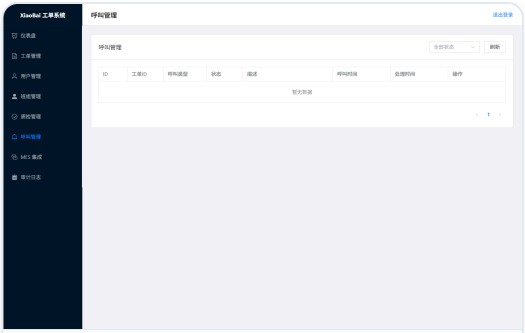
工人遇到异常时触发呼叫，管理员实时响应并处理 · 角色：工人 + 管理员

D1. 工人触发呼叫（三类）

工人端 CallView 提供3种呼叫类型：

呼叫类型	使用场景	接口
Andon	生产线异常停机、设备故障	POST /api/device/call/andon
质检呼叫	需要质检员到现场检查	POST /api/device/call/inspection
搬运呼叫	需要物料搬运/转序	POST /api/device/call/transport

工人端呼叫页 — 选择呼叫类型 + 描述



管理端呼叫列表 — PENDING/HANDLING/HANDLED 状态

D2. 管理员处理呼叫

1 呼叫触发

工人点击呼叫按钮，系统创建 CallRecord，状态 PENDING

▼

2 管理员接单

管理员在 /calls 页面看到新呼叫，点击"处理"，状态变为 HANDLING，记录处理人

▼

3 **处理完成**

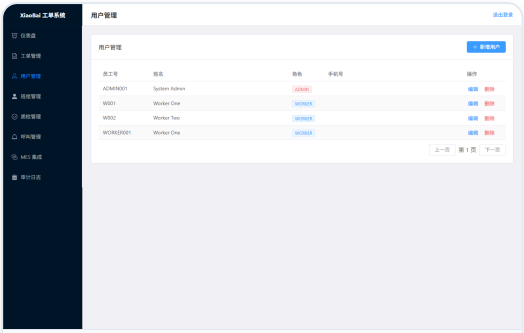
管理员填写处理结果，点击"完成"，状态变为 HANDLED，记录完成时间

⚠ **当前限制：无实时推送**

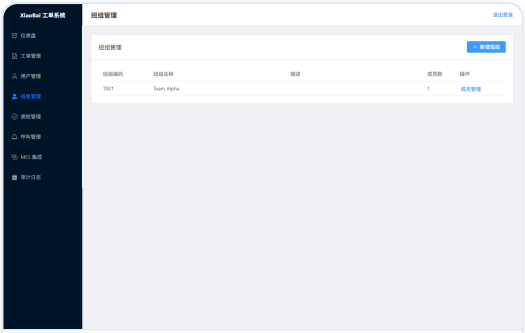
管理员需手动刷新页面才能看到新呼叫。系统尚未实现 WebSocket/SSE 推送，Andon 场景的时效性依赖人工轮询。

06 管理后台功能全览

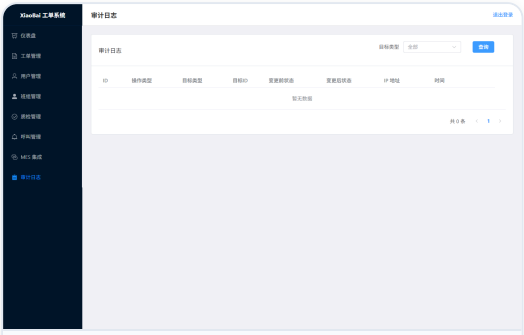
用户管理、班组管理、审计日志、MES 集成



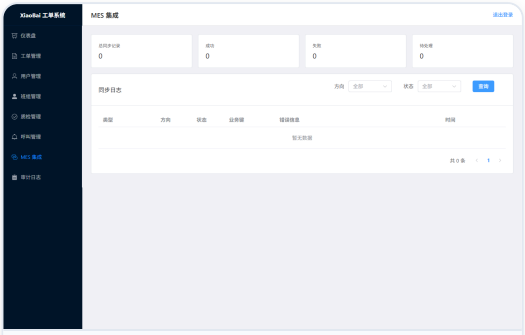
用户管理 — 创建/编辑/软删除用户，分配 ADMIN/WORKER 角色



班组管理 — 创建班组、添加/移除成员



审计日志 — ISO 9001 合规，AOP 自动记录操作，支持按用户/操作类型筛选



MES 集成监控 — 同步统计（成功/失败/重试）+ 同步日志查询

MES 双向集成架构

入 MES → 易工单（Webhook 入站）

POST /api/mes/work-orders/import — MES 推送工单，自动创建工单+工序+分配，mesOrderId 幂等

出 易工单 → MES（定时出站推送）

MesSyncScheduler 定时将报工/状态/质检结果推送回 MES，指数退避重试，最多5次

⚠ 当前状态

> MES 集成代码已完整实现，但无真实 MES 系统对接。出站推送在无 MES 环境时会持续重试并累积日志。

07 关键风险与缺口

架构风险 · 代码质量 · 功能缺口 · 建议下一步行动

🔴 高优先级风险

R1 • JWT Secret 无默认值

APP_JWT_SECRET 环境变量缺失时服务启动失败（这是有意的 Fail Fast 设计）。CI/CD 流水线和生产部署必须注入该变量，建议使用 Kubernetes Secret 或 Vault 管理。

🟡 中优先级风险

R2 • 无实时推送 (WebSocket/SSE)

Andon 呼叫等时效性强的场景，管理员需手动刷新页面。建议引入 WebSocket 或 SSE 实现实时通知。

R3 • DeviceController 过于臃肿

工人端所有操作（报工、扫码、质检、呼叫、返工）集中在一个 Controller，约 20+ 方法。建议按业务拆分为独立 Controller。

R4 • statusLabel.js 双份维护

Admin 和 Worker 两个项目各有一份状态标签映射文件，需同步维护。建议提取为共享包或 monorepo 工具模块。

R5 • 工序依赖疑似缺少环检测

AddDependency 接口在添加依赖时疑似未检测循环依赖。若形成环路，Kahn 拓扑排序将陷入死循环。需人工确认并添加 DFS 环检测。

🟢 功能缺口

G1 • 工人产出统计接口疑似缺失

前端 api/statistics.js 中有 getWorkerOutput 调用，但后端 /api/admin/statistics/worker-output 接口待确认是否实现。

G2 • 无工单模板/复制功能

高频创建相同产品工单时，需每次手填。建议增加"复制工单"或"工单模板"功能。

G3 • 权限粒度较粗

目前仅 ADMIN / WORKER 两个角色，无法实现"只能操作自己班组的工单"等细粒度控制。

建议下一步行动

优先级	行动项	说明
🔴 立即	验证 Docker 完整启动	执行 docker-compose up -d，全链路端到端测试
🔴 立即	确认 worker-output 接口	检查 /api/admin/statistics/worker-output 是否实现
🟡 短期	添加工序依赖环检测	AdminOperationDependencyController#add 中加 DFS 检测
🟡 短期	拆分 DeviceController	按扫码/报工/呼叫/质检拆分为4个 Controller
🟡 中期	引入 WebSocket/SSE	为 Andon 呼叫实现实时推送
🟢 长期	TypeScript 迁移	前端引入 TS 提升类型安全

08 架构改进建议：Agent / MCP / Skills 视角

评估引入现代 AI 工程架构的可行性与落地路径

8.1 将"代码调研"封装为可复用 Skill ✅ 强烈推荐

本次调研流程本身就是一个高度结构化、可重复执行的工作流，完全可以封装为 `code-audit-skill`：

```
Skill: code-audit
  输入: repo_path, output_dir, run_mode (static | with_browser)
  子步骤（可并行）:
    1. TechStackScanner     → 识别框架/工具链 → tech_stack.json
    2. RouteAPIMapper        → 提取路由+接口   → api_map.json
    3. FeatureExtractor     → 推断功能点       → features.json
    4. BrowserAuditor       → 页面截图         → screenshots/
    5. ReportGenerator      → 合并生成报告     → report.md + report.pdf
  输出: current_state_report.md, current_state_report.pdf, screenshots/
```

8.2 Multi-Agent 并行子任务流 ✅ 推荐（大型仓库效果明显）

Agent	工具	职责
Repo Scanner	Glob, Grep, Read	文件结构、技术栈识别
Route/API Mapper	Read (controllers/routers)	接口路径、参数提取
Browser Auditor	Bash, Playwright MCP	前端启动、截图、页面验证
Report Generator	Write, PDF MCP	汇总所有 JSON，生成报告

8.3 MCP 接入建议

MCP/插件	用途	优先级
Playwright MCP	浏览器截图、页面导航、表单填写	🔴 P1（解决当前手动截图问题）

MCP/插件	用途	优先级
PDF Export MCP	Markdown → PDF 高质量转换	🟡 P2
知识库/向量库 MCP	历史调研存档、版本对比	🟡 P2（适合迭代项目）
GitHub MCP	直接读取 PR/Issue/代码	🟢 P3

8.4 对于易工单业务本身 — 部分适合

✅ **适合引入：智能派工助手**

基于工人历史绩效、当前负荷、技能标签，AI 推荐最优派工方案。可作为 LLM 调用层接入 /api/admin/work-orders/assign。

💡 **中期可考虑：质量预测**

基于历史质检记录，预测当前工序的质量风险，提前提示质检员重点关注。

⚠️ **不建议：将核心状态机替换为 LLM**

工单状态机逻辑明确、规则固定，LLM 引入会增加不确定性和延迟。当前 WorkOrderStateMachine 设计已足够健壮。